IKOSOFT / 42 - 1337 / SUBJECT VERSION 2.0

RAG against the machine

20-day validation and recoding roadmap

Prepared for yikoubaz | 12-31 October 2026

Your target: build a compliant RAG system, prove its retrieval scores and performance, then rebuild the essential pipeline from a blank directory with understanding.

Phase	Days	Evidence you must produce
Foundations + chunking	1-6	Validated contracts, reproducible CLI, exact source spans
Lexical retrieval + evaluation	7-12	Persistent BM25 index, batch search, measured recall
Grounded generation + hardening	13-17	Qwen answers, robust inputs, official validation gates
Rebuild + defense	18-20	Two independent rebuilds, peer review, clean submission

Work rhythm: 8 focused hours per day

1 h learn and sketch; 4 h implement; 1.5 h test and measure; 1 h recode without looking; 0.5 h explain, record results and commit. Take breaks outside these blocks. This is a 160-hour planning assumption, not a promise of passing.

How this fits your learning style

Use your Call Me Maybe experience for local-model integration and Pydantic. Spend the most effort on chunking, ranking and evaluation. Each day ends with a working artifact, a measurable check and a small rebuild from memory.

Dates assume you start today. Keep the day numbers if your start moves. Full-pipeline rebuilding is your personal mastery goal; the subject describes brief recode modifications during evaluation, not a mandatory full rebuild (X.1, pp. 24-25).

01 / SUBJECT CONTRACT

What validation requires

Requirement	Exact target / evidence
Environment	Python 3.10+, uv project management, root pyproject.toml + uv.lock; uv sync must work.
Interface	Python Fire; all six commands; configurable paths; tqdm for long operations.
Index	Provided vLLM corpus; distinct Python and Markdown/text chunkers; index persisted in data/processed/.
Retrieval	Implement TF-IDF or BM25. This plan chooses BM25 and has you rebuild its scoring yourself.
Sources	Exact corpus-relative file_path; original character ranges; every returned source at most 2000 characters.
Scores	At least 80% docs recall@5 and 50% code recall@5.
Speed	Whole-corpus indexing at most 5 min; retrieval for 200 questions at most 90 s.
Answers	Qwen/Qwen3-0.6B must work; coherent, on-topic, mostly grounded answers; correct Pydantic JSON.
Quality	flake8, typed functions, mypy, PEP 257 docstrings, resource cleanup and graceful errors.
Submission	src/, root README in English, Makefile install/run/debug/clean/lint, dependency files and necessary config. No large corpus, weights or generated outputs committed.

Exact lint commands

```
flake8 .
```

```
mypy . --warn-return-any --warn-unused-ignores
--ignore-missing-imports --disallow-untyped-defs
--check-untyped-defs
```

Source: rag.pdf v2.0, V pp. 8-9; VI pp. 10-18; VII pp. 19-20; VIII pp. 21-22; X pp. 24-25. Printed subject page numbers are used throughout.

02 / DESIGN BEFORE CODING

Keep the pipeline explainable

Suggested module	One responsibility
models.py	Required Pydantic input/output contracts + internal Chunk model
ingest.py / chunking.py	Read original text, preserve path, return exact bounded spans
tokenize.py / bm25.py	Identifier-aware tokens, term statistics, lexical ranking
index.py / storage.py	Build and persist chunks/statistics; reload in a new process
retrieval.py	Query index, rank sources, apply k consistently
generation.py	Load Qwen, pack bounded context, produce answer text
evaluation.py	Independent recall calculation for iteration
cli.py / __main__.py	Fire commands, orchestration, error messages and JSON output

Data flow

Raw files → exact source chunks → tokens and BM25 statistics → persisted index.
Question → query tokens → ranked MinimalSource records → original snippets → bounded model prompt → answer.
Dataset → search-results JSON → external official score → answer-results JSON.

Span invariant: preserve text, normalize only for scoring

Adopt [start, end) internally so chunk_text == original_text[start:end]. The subject names start/end indices but does not explicitly settle endpoint inclusivity. Check provided examples and official validation early; adapt serialization if needed. Python AST column offsets are UTF-8 byte offsets: convert carefully before computing Python string character indices.

Repository layout

```
src/ | pyproject.toml | uv.lock | Makefile | README.md
data/raw/vllm-0.10.1/ | data/processed/
data/datasets/UnansweredQuestions/ | AnsweredQuestions/
data/output/search_results/<DatasetScope>/
data/output/search_results_and_answer/<DatasetScope>/
```

Suggested architecture, not mandated module names. Paths/layout follow VI.7, pp. 16-18. Index configuration should travel with the persisted index so query-time tokenization matches indexing.

DAILY EXECUTION / DAYS 01-02

Build, verify, then rebuild

DAY 01 | 12 Oct | Foundations

Understand the contract and trace one question

LEARN Read the whole subject. Explain indexing, retrieval, augmentation and generation; distinguish a character limit from a token budget.

BUILD Create a requirement checklist and module sketch. Obtain the provided corpus, datasets and official executable. Trace a docs question from its reference source to the original file; inspect one code question too.

VERIFY Check dataset shapes, preserved question IDs, actual corpus prefix and sample spans. Record hardware and available storage. Identify any missing assets immediately.

RECODE From memory, describe the entire pipeline and write the three fields of MinimalSource. Recreate one source slice by hand.

DONE WHEN A one-page contract checklist and two source traces exist. You can explain why a correct idea in the wrong file scores zero.

DAY 02 | 13 Oct | Foundations

Bootstrap the CLI and test Qwen early

LEARN Review uv, Python Fire and Pydantic. Inspect the available Qwen loading interface; keep model integration behind a small adapter.

BUILD Create src/__main__.py, all six Fire command entry points, pyproject.toml and uv.lock. Add Makefile targets and .gitignore. Load Qwen/Qwen3-0.6B and run one tiny prompt on the target machine.

VERIFY Run uv sync and CLI help in a new shell. Confirm import paths, dependencies, model cache location and enough disk space. Record model startup time.

RECODE Rebuild a minimal Fire CLI with one typed command and a Pydantic result without copying the project.

DONE WHEN Commands are discoverable and the required model loads. Resolve installation or hardware blockers before building more stages.

DAILY EXECUTION / DAYS 03-04

Build, verify, then rebuild

DAY 03 | 14 Oct | Foundations

Implement models and scan original files

LEARN Understand question inheritance, dataset unions, source ranges, context-managed file reads and validation errors.

BUILD Implement all eight named subject models. Add an internal chunk record. Scan useful .py, Markdown and text files deterministically; retain the exact data/raw/... prefix and unmodified decoded text.

VERIFY Round-trip sample search and answer JSON through Pydantic. Keep existing question IDs. Check non-ASCII text, unreadable files, empty files and no accidental path prefix changes.

RECODE Recreate MinimalSource, MinimalSearchResults and StudentSearchResults plus a deterministic file scanner.

DONE WHEN Models match VI.4-5. The scanner reports corpus counts and produces exact reproducible source paths.

DAY 04 | 15 Oct | Chunking

Build Markdown and text chunking

LEARN Study headings, paragraphs, fenced blocks, overlap and how a hard maximum differs from a preferred boundary.

BUILD Segment Markdown by structure, then split oversized sections by paragraphs/lines with a bounded fallback. Add a text strategy. Store start/end indices into the untouched source; make size configurable up to 2000.

VERIFY For every test chunk assert exact slicing and the size cap. Exercise long lines, long fences, Unicode, tiny limits and empty text. Check coverage and termination.

RECODE Write a bounded span splitter from scratch and explain the loop invariant that prevents infinite overlap loops.

DONE WHEN Markdown/text chunks preserve original content and never exceed the requested legal size. Reject invalid sizes clearly.

DAILY EXECUTION / DAYS 05-06

Build, verify, then rebuild

DAY 05 | 16 Oct | Chunking

Build Python chunking without losing offsets

LEARN Study Python AST node boundaries, decorators, nested definitions, line-start tables and byte-versus-character column offsets.

BUILD Chunk functions/classes and useful module-level content. Include decorators where appropriate. Split large nodes and retain imports/docstrings as retrievable content. Add a safe line-based fallback for parse errors.

VERIFY Exercise multiline decorators, nested classes, oversized functions, Unicode before nodes and syntax errors. Reconstruct each chunk from the original source using its stored span.

RECODE Rebuild the line-offset conversion and a function-span extractor; explain why AST columns cannot always be used as character counts.

DONE WHEN A genuinely distinct Python strategy works, with bounded fallback chunks and no dropped useful module regions.

DAY 06 | 17 Oct | Chunking

Persist a complete first index

LEARN Understand index versioning, configuration consistency and disk serialization. Separate retrieval metadata from output models.

BUILD Connect scanner and both chunkers to index --max_chunk_size. Persist chunks and metadata under data/processed/. Add tqdm, useful counts and readable errors. Store tokenizer configuration and corpus information.

VERIFY Index the full corpus and time it. Reload in a separate process; check paths, spans, counts and deterministic ordering. Inspect all chunk lengths. Establish a performance baseline.

RECODE Rebuild read → chunk → save → reload on a tiny corpus without the existing source code.

DONE WHEN The complete corpus is indexed and reloadable. Day 6 timing is a baseline; the final requirement remains at most 300 seconds.

DAILY EXECUTION / DAYS 07-08

Build, verify, then rebuild

DAY 07 | 18 Oct | Retrieval

Tokenize code and learn BM25 numerically

LEARN Study term frequency, document frequency, IDF, length normalization, k1 and b. Compute one ranking on a three-document toy corpus.

BUILD Build a shared query/document tokenizer. Preserve identifier tokens and optionally add snake_case/camelCase parts; handle punctuation and case consistently. Compute per-chunk term counts, lengths, document frequencies and average length.

VERIFY Check tokens for add_lora, AsyncLLMEngine, flags and natural-language questions. Manually verify statistics and a BM25 score. Retain rare identifiers; avoid aggressive stopword removal initially.

RECODE Reimplement tokenization and document frequency, then calculate the ranking without looking at your implementation.

DONE WHEN You can explain every BM25 term and show how identifier handling changes which chunks can match.

DAY 08 | 19 Oct | Retrieval

Implement top-k single-query retrieval

LEARN Learn inverted postings, candidate accumulation, deterministic ties and top-k selection. Understand why scores are not probabilities.

BUILD Implement BM25 scoring yourself using postings and per-document statistics. Expose search with ranked MinimalSource outputs. Load the index once per operation and define consistent k handling.

VERIFY Compare the optimized ranker to a simple full-scan oracle on a toy corpus. Test empty query, unknown terms, k=0, negative k and k larger than available results.

RECODE Rebuild score accumulation and top-k selection from memory. Explain the ranking cost and the role of each statistic.

DONE WHEN Single search returns correct bounded sources with stable ordering; malformed inputs produce clear messages without unhandled tracebacks.

DAILY EXECUTION / DAYS 09-10

Build, verify, then rebuild

DAY 09 | 20 Oct | Retrieval

Search datasets and verify official format

LEARN Study batch orchestration, JSON serialization, dataset scope and how repeated file names can overwrite outputs.

BUILD Implement search_dataset with configurable dataset_path, k and save_directory. Preserve question_id and question; write StudentSearchResults with the requested k. Load one index for the whole batch; add tqdm.

VERIFY Run docs and code datasets into separate scoped directories. Validate every source. Use the provided official executable manually to check output validity and early recall; never call/import it from your solution.

RECODE Rebuild a batch search wrapper that reads questions, retrieves sources and writes validated JSON.

DONE WHEN Public search outputs pass official structural validation. Record docs/code recall and batch runtime as your first real baseline.

DAY 10 | 21 Oct | Evaluation

Implement recall and diagnose retrieval misses

LEARN Learn recall@k per question: matched reference sources divided by reference source count. Match by exact file and span IoU threshold 0.05; inspect official behavior for boundary/empty cases.

BUILD Implement evaluate independently. Match questions by ID; count each reference source once even if several retrieved chunks hit it. Report macro-average recall and produce a miss-analysis table for docs and code.

VERIFY Test wrong file, disjoint spans, small overlap, duplicate hits and multiple reference sources. Compare local scores with the official executable and investigate discrepancies before tuning.

RECODE Rebuild interval intersection/union and per-question recall from hand-written examples.

DONE WHEN Local evaluation is trustworthy enough for iteration. Classify at least 20 misses as coverage, tokenization, ranking, span or output-contract failures.

DAILY EXECUTION / DAYS 11-12

Build, verify, then rebuild

DAY 11 | 22 Oct | Evaluation

Tune document retrieval with evidence

LEARN Study controlled experiments: change one factor, compare the same queries and avoid fitting directly to reference locations.

BUILD Try a small matrix of chunk sizes/overlap and heading-aware scoring. Optionally add light path/title tokens while preserving original spans. Record each configuration, recall and runtime; retain a baseline.

VERIFY Inspect before/after rankings on representative docs misses. Keep a development/holdout split if dataset size permits. No hard-coded question answers or reference-source lookup in retrieval.

RECODE Rebuild heading segmentation and one configurable ranking adjustment; explain a case where overlap hurts top-5 diversity.

DONE WHEN Choose the strongest measured docs configuration. Aim to reach at least 0.80 docs recall@5 and record remaining misses if below it.

DAY 12 | 23 Oct | Evaluation

Tune code retrieval and lock a candidate

LEARN Understand lexical mismatch between behavior descriptions and identifiers, and redundant chunks competing in top-5.

BUILD Experiment with identifier splitting, imports/docstrings, chunk boundaries and modest symbol/path weighting. Test deduplication/diversity only when measured. Run both docs and code after each retained change.

VERIFY Run official validation at k=5. Target docs at least 0.80 and code at least 0.50 without breaking the 2000-character cap. Time 200 questions when available; record dataset, machine and measurement method.

RECODE Rebuild tokenizer → BM25 → sources on a tiny mixed corpus. Explain your final defaults and rejected changes.

DONE WHEN A measured retrieval candidate is saved. If thresholds fail, reserve optional polish time for the largest miss categories; do not claim readiness.

DAILY EXECUTION / DAYS 13-14

Build, verify, then rebuild

DAY 13 | 24 Oct | Generation

Pack context safely for the small model

LEARN Understand token budgets, chat formatting, prompt overhead and reserving space for the output. Characters do not equal model tokens.

BUILD Recover text from retrieved spans. Label snippets with their file paths. Build a prompt that asks for source-grounded answers and acknowledgment of missing evidence. Pack context using the actual tokenizer and model limit.

VERIFY Check no-context, conflicting snippets and a full context window. Ensure instructions/question plus evidence plus reserved generation tokens fit. Treat retrieved code/comments as evidence, not instructions.

RECODE Rebuild a context packer with a mock tokenizer, then explain where the generation-token reserve enters the calculation.

DONE WHEN The prompt and retrieved context fit the configured Qwen budget. Useful evidence is retained in ranked order and missing evidence is handled.

DAY 14 | 25 Oct | Generation

Generate single grounded answers

LEARN Review the Qwen adapter, decoding controls and how retrieval failures differ from generation failures.

BUILD Implement answer through the same retriever and context builder. Use Qwen/Qwen3-0.6B; place its answer string in the required structured output. Validate JSON in Python rather than relying on the model to invent the schema.

VERIFY Inspect at least 10 docs/code answers against their snippets. Log hallucinated claims and prompt changes. Check empty evidence, initialization failure and output length; release resources appropriately.

RECODE Rebuild retrieve → context → model adapter → validated answer using a fake model, then swap in Qwen.

DONE WHEN Single-query answers are mostly grounded and understandable. Model wording may be imperfect; retrieval quality and grounding remain the priority.

DAILY EXECUTION / DAYS 15-16

Build, verify, then rebuild

DAY 15 | 26 Oct | Generation

Generate dataset answers from saved results

LEARN Study the difference between answering a new query and using previously saved search results.

BUILD Implement answer_dataset with student_search_results_path and save_directory. Consume saved sources directly, preserve IDs/questions/k, reuse one model instance and write StudentSearchResultsAndAnswer JSON with tqdm.

VERIFY Run the subject sequence: index → search_dataset → external score → answer_dataset. Check scoped output names, original source ranges, valid JSON and one result per input question.

RECODE Rebuild the saved-results answer loop with a stub model. Explain why this command must not silently replace retrieval results.

DONE WHEN All six commands work end to end. Search and answer artifacts comply with their respective contracts.

DAY 16 | 27 Oct | Validation

Harden inputs and measure the real bottleneck

LEARN Review validation boundaries, useful error messages and profiling. Catch specific expected failures without hiding programming defects.

BUILD Handle missing/corrupt index, missing files, malformed JSON, invalid ranges and nonsensical queries. Run flake8/mypy. Profile indexing and batch retrieval; remove repeat loads and unnecessary full scans.

VERIFY Exercise every listed subject edge case across relevant commands. Time a clean index and retrieval batch separately from generation. Verify non-default paths and repeated runs; keep clean from deleting raw inputs.

RECODE Add a new CLI option and rebuild a clear file-validation/error boundary in a few minutes.

DONE WHEN No tested degenerate input yields an unhandled traceback. Measured speed and lint results are written down with remaining blockers.

DAILY EXECUTION / DAYS 17-18

Build, verify, then rebuild

DAY 17 | 28 Oct | Validation

Run a complete evaluation rehearsal

LEARN Learn how to present reproducible evidence: corpus version, datasets, hardware, configuration, command and score.

BUILD Run from a clean checkout with data/weights provisioned separately. Execute uv sync, Makefile targets, the official flow, custom evaluate and single-query commands. Ask a peer to drive the CLI and question design choices.

VERIFY Require official-valid results, docs recall@5 at least 80%, code at least 50%, index at most 5 min and 200-query retrieval at most 90 s. Inspect grounding and verify no required path is hard-coded.

RECODE Peer requests three small changes: tie-break, output field via an internal model, and chunk-size setting. Explain and implement each.

DONE WHEN Keep a pass/fail matrix. Any failed mandatory gate becomes the top task; postpone extras rather than masking the failure.

DAY 18 | 29 Oct | Recode

Rebuild the retrieval core from a blank folder

LEARN Review only your one-page architecture, then close implementation files and AI-generated solutions. Use library/API documentation as needed.

BUILD In an isolated scratch directory, rebuild models, scanner, both chunkers, tokenizer, BM25 statistics/scoring, persistence and search. Start with a tiny corpus; expand only after correctness checks.

VERIFY Use your independent toy fixtures to check exact spans, reload, ranking and top-k. Compare behavior with the main project after completing the attempt. Log what you could not reconstruct.

RECODE This entire day is a retrieval rebuild rehearsal. Spend about 6 h rebuilding and 2 h testing/explaining; prioritize correctness over typing speed.

DONE WHEN A working standalone retrieval core exists. If incomplete, identify exact gaps and rebuild those modules again before the final rehearsal.

DAILY EXECUTION / DAYS 19-20

Build, verify, then rebuild

DAY 19 | 30 Oct | Recode

Rebuild orchestration and teach the design

LEARN Explain each stage aloud: inputs, outputs, invariants, failure modes and complexity. Review notes only after attempting the explanation.

BUILD Extend the new core with Fire, dataset search, independent recall, context packing and answer commands. Use a stub model first and then the installed Qwen adapter. Draft the English README from actual measurements.

VERIFY Check all six rebuilt commands on tiny fixtures and at least one real question. Have a peer trace a random question and request a minor behavior change. Re-run lint on the submitted implementation.

RECODE Timed drills: reconstruct BM25, IoU recall, a span splitter and one batch wrapper in separate 15-30 minute attempts.

DONE WHEN The rebuilt pipeline runs end to end. You can teach its data flow and explain every formula and schema without opening the original code.

DAY 20 | 31 Oct | Recode

Final rebuild checkpoint and submission audit

LEARN Review only your gap list. Focus on demonstrated understanding and reproducible mandatory results.

BUILD Do a second independent minimal end-to-end rebuild: read/chunk → index → retrieve → context → generate → evaluate, with JSON and persistence. Then audit and freeze the real submission after final fixes.

VERIFY Repeat the Day 17 gates if code changed. Verify README sections, exact lint flags, clean clone setup and required files. Inspect git tracking: exclude large corpus, model weights and generated outputs.

RECODE Explain one question through every intermediate object. Make one unprepared peer-requested change and rerun the affected checks.

DONE WHEN Submit only when every mandatory gate is evidenced. Personal mastery means an independent working rebuild and explanations; unfinished rebuild gaps remain explicit learning tasks.

13 / REPRODUCIBLE RUNBOOK

Use the evaluator-compatible flow

Run from the repository root. Commands below use the subject example paths; your CLI must accept evaluator-supplied paths. Use k=5 for the required recall target; k=10 is also shown in the subject walkthrough.

1. Install and index

```
uv sync
uv run python -m src index --max_chunk_size 2000
```

2. Search docs (repeat for code dataset)

```
uv run python -m src search_dataset
--dataset_path data/datasets/UnansweredQuestions/dataset_docs_public.json
--k 5
--save_directory data/output/search_results/UnansweredQuestions
```

3. Score externally; never from solution code

```
./moulinette evaluate_student_search_results
data/output/search_results/UnansweredQuestions/dataset_docs_public.json
data/datasets/AnsweredQuestions/dataset_docs_public.json
--k 5 --max_context_length 2000
```

4. Generate from saved search results

```
uv run python -m src answer_dataset
--student_search_results_path
data/output/search_results/UnansweredQuestions/dataset_docs_public.json
--save_directory data/output/search_results_and_answer/UnansweredQuestions
```

5. Independently evaluate for debugging

```
uv run python -m src evaluate
--student_search_results_path
data/output/search_results/UnansweredQuestions/dataset_docs_public.json
--dataset_path data/datasets/AnsweredQuestions/dataset_docs_public.json
```

6. Exercise single queries

```
uv run python -m src search "How is LoRA configured?" --k 5
uv run python -m src answer "How is LoRA configured?" --k 5
```

Multiline commands are displayed for readability. Enter as one shell command, or use shell line-continuation backslashes. Rename the provided OS-specific moulinette executable to moulinette first. Source: VI.6-7, pp. 16-18.

14 / MASTERY REFERENCE

Formulas and recode drills

BM25: one defensible implementation choice

```
score(q, d) = sum over query terms t of:
IDF(t) * tf(t,d)*(k1+1) /
(tf(t,d) + k1*(1-b + b*len(d)/avg_len))
```

A common positive IDF is ln(1 + (N-df(t)+0.5)/(df(t)+0.5)). N is chunk count; df counts chunks containing t; tf counts t in a chunk. k1 controls frequency saturation and b length normalization. Begin with k1=1.5 and b=0.75 as experimental defaults. BM25 variants and these constants are design choices, not subject requirements.

Recall and span overlap

```
intersection = max(0, min(end_a,end_b) - max(start_a,start_b))
union = length_a + length_b - intersection
IoU = intersection / union
recall_q@k = matched_reference_sources / total_reference_sources
```

Match only identical file paths, with the subject IoU bar of 0.05. A reference source counts once even if hit by multiple results. Average question recalls; confirm exact threshold comparison, endpoint convention and zero-source behavior against supplied material/official scores. Example: 2 of 3 reference sources matched gives 2/3 recall, regardless of duplicate hits. Source: VII.1, p. 19.

Drill	Independent acceptance check
Span splitter / 15 min	Correct original slices; never over cap; terminates on a long line
Tokenizer / 15 min	Same rules at indexing/query time; identifier examples work
BM25 / 30 min	Ranking equals a hand-computed three-document example
Persistence / 20 min	A new process reloads and returns the same ranking
Recall / 20 min	Multiple sources, duplicate hits and wrong-file cases work
Context packer / 20 min	Prompt + evidence + answer reserve fit the model limit
CLI change / 10 min	New option propagates correctly; affected checks pass

Rebuild rules

Attempt first without source code, AI solutions or copy-paste. External dependency documentation is fine. Afterward compare, list gaps, fix them and repeat later. The goal is rebuilding your own logic, not rewriting Fire, Pydantic or the model runtime.

15 / FINAL ACCEPTANCE

Evidence before submission

Gate	Record / check
[ ] Official format	All emitted files validate; exact paths, bounded spans, correct IDs and k.
[ ] Retrieval quality	Docs recall@5 >= 0.80; code recall@5 >= 0.50. Record official executable results and dataset names.
[ ] Performance	Full index <= 300 s; 200-query retrieval <= 90 s on evaluation-like hardware. Do not include answer generation in retrieval timing.
[ ] All six commands	index, search, search_dataset, answer, answer_dataset, evaluate; non-default paths work.
[ ] Grounding	Qwen/Qwen3-0.6B works; inspect answers against evidence and show bounded prompts.
[ ] Robustness	Empty/nonsensical queries, k=0, missing files and malformed JSON handled; no unhandled tracebacks.
[ ] Code quality	flake8 and required mypy flags pass; type hints, PEP 257 docstrings and safe resource handling.
[ ] Reproducibility	Root uv sync works; correct layout; Makefile install/run/debug/clean/lint work.
[ ] README	English; italicized first line; Description, Instructions, Resources + AI use; architecture, chunking, retrieval, performance, choices, challenges, examples.
[ ] Submission hygiene	Implementation/config in Git; large data, weights and generated outputs excluded.
[ ] Personal recode goal	Independent minimal full rebuild works; explain BM25, offsets, recall, context and error handling; handle an unprepared small change.

README first line

```
*This project has been created as part of the 42 curriculum by yikoubaz.*
```

If you fall behind

Fix invalid outputs before score tuning. Fix corpus coverage and tokenizer issues before adding complex ranking. Use Days 16-17 for unresolved mandatory gates and repeat the affected checks. Keep bonuses out of this 20-day schedule; the subject grades them only after the mandatory part validates in full. A missed gate calls for more work, not an assumed pass.

Source map: V pp. 8-9 (quality/tooling); VI pp. 10-18 (pipeline/contracts); VII pp. 19-20 (metrics/speed); VIII pp. 21-22 (README); IX p. 23 (bonus); X pp. 24-25 (submission/recode). Roadmap timing, module choices and personal rebuild drills are recommendations.